HUMAN AWARE RE-IDENTIFICATION AND TRACKING

A Project Progress Report

Submitted in partial fulfillment for the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

Submitted by

K.HARSHITHA	(N200373)
CH.RAJESWARI	(N200427)
B.CHETHAN	(N200518)
SK.AKHIL	(N200145)

Under the Esteem Guidance of

Y.KALAVATHI

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Rajiv Gandhi University of Knowledge Technologies – Nuzvid

Nuzvid, Eluru, Andhra Pradesh – 521202

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Rajiv Gandhi University of Knowledge Technologies – Nuzvid

Nuzvid, Eluru, Andhra Pradesh – 521202

CERTIFICATE OF COMPLETION

This is to certify that the work entitled, **"HUMAN AWARE RE-IDENTIFICATION AND TRACKING"** is the bonafide work of **K.HARSHITHA (N200373), CH.RAJESWARI (N200427), B.CHETHAN (N200518), SK.AKHIL (N200145)** carried out under my guidance and supervision for 4th year project of Bachelor of Technology in the department of Computer Science and Engineering under RGUKT IIIT Nuzvid. This work is done during the academic session August 2025 – April 2026, under our guidance.

_____

Y.Kalavathi
Mentor,
Department of CSE,
RGUKT Nuzvid.

_____

A.Uday Kumar
Assistant Professor,
Head of the Department,
Department of CSE,
RGUKT Nuzvid.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Rajiv Gandhi University of Knowledge Technologies – Nuzvid

Nuzvid, Eluru, Andhra Pradesh – 521202

CERTIFICATE OF EXAMINATION

This is to certify that the work entitled,**"HUMAN AWARE RE-IDENTIFICATION AND TRACKING"** is the bonafied work of **K.HARSHITHA (N200373), CH.RAJESWARI (N200427), B.CHETHAN (N200518), SK.AKHIL (N200145)** and here by accord our approval of it as a study carried out and presented in a manner required for its acceptance in 4th year of **Bachelor of Technology** for which it has been submitted. This approval does not necessarily endorse or accept every statement made, opinion expressed or conclusion drawn, as a recorded in this thesis. It only signifies the acceptance of this thesis for the purpose for which it has been submitted.

_____

Y.Kalavathi
Mentor,
Department of CSE,
RGUKT-NUZVID

_____

Project Examiner
RGUKT-Nuzvid

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Rajiv Gandhi University of Knowledge Technologies – Nuzvid

Nuzvid, Eluru, Andhra Pradesh – 521202

DECLARATION

We, **K.HARSHITHA (N200373), CH.RAJESWARI (N200427), B.CHETHAN (N200518), SK.AKHIL (N200145)** hereby declare that the project report entitled **"HUMAN AWARE RE-IDENTIFICATION AND TRACKING"** done by us under the guidance of Y.Kalavathi, is submitted for the partial fulfillment of the award of degree of Bachelor of Technology in Computer Science and Engineering during the academic session August 2025 - April 2026 at RGUKT-Nuzvid.

We also declare that this project is a result of our own effort and has not been copied or imitated from any source. Citations from any websites are mentioned in the references. The results embodied in this project report have not been submitted to any other university or institute for the award of any degree or diploma.

Date: 10-04-2026	K.HARSHITHA	(N200373)
Place: Nuzvid	CH.RAJESWARI	(N200427)
	B.CHETHAN	(N200518)
	SK.AKHIL	(N200145)

ACKNOWLEDGEMENT

We would like to express our profound gratitude and deep regards to our guide **Y.Kalavathi** for his exemplary guidance, monitoring, and constant encouragement throughout the B.Tech course. We shall always cherish the time spent with him during the course of this work due to the invaluable knowledge gained in the field of reliability engineering.

We are extremely grateful for the confidence bestowed upon us and for entrusting us with our project entitled **"HUMAN AWARE RE-IDENTIFICATION AND TRACKING"**.

We express our gratitude to **Mr.A.Uday Kumar** (HOD of CSE) and other faculty members for being a source of inspiration and constant encouragement, which helped us in completing the project successfully.

Our sincere thanks go to all the batchmates of 2020 CSE, who have made our stay at RGUKT-NUZVID a memorable one.

Finally, yet importantly, we would like to express our heartfelt thanks to our beloved God and parents for their blessings, and our friends for their help and wishes for the successful completion of this project.

CHAPTER 1
ABSTRACT

Human tracking and biometric identification have become essential components in modern intelligent surveillance and attendance management systems. With increasing reliance on automated systems in academic and institutional environments, there is a strong need for solutions that can accurately detect, track, and identify individuals in real-time from live video streams or pre-recorded footage, even under conditions of occlusion, re-entry, pose variation, and lighting change.

This project presents a real-time biometric attendance management system that integrates computer vision, deep learning, and a web-based interface to fully automate student attendance in classroom environments. The system combines multi-object person detection, persistent tracking, and face recognition into a unified pipeline capable of processing both live webcam feeds and uploaded MP4 recordings through a single platform.

Person detection is performed using YOLOv11n, a lightweight single-stage detector, and each detected individual is assigned a persistent identity across frames using the BoT-SORT multi-object tracker with Kalman filter-based motion prediction. Face recognition is handled by InsightFace (Buffalo_L model), which extracts 512-dimensional facial embeddings for identity matching via cosine similarity. A two-tier matching strategy is employed: a dynamic session cache storing up to five diverse embeddings per student enables fast in-session recognition, while a persistent per-section database serves as the fallback for new detections.

Recognition reliability is ensured through a consensus-based locking mechanism that requires agreement across multiple frames before confirming an identity, reducing false positives from single-frame noise. Adaptive shadow updates continuously refine session embeddings in the background to account for changes in lighting and pose over the course of a session. Unknown individuals who cannot be matched are detected, logged with captured snapshots, and tracked separately as visitors.

Attendance is recorded using a heartbeat-based interval logging mechanism that fires every ten seconds, synchronising the database and maintaining continuous entry and exit timestamps for each student. This enables computation of precise attendance durations, detection of mid-session absences, and generation of per-student time-in-room analytics. The system is backed by a Django web dashboard providing live MJPEG video streaming, real-time attendance monitoring, session controls, and a reports interface with CSV export.

The system incorporates an SLM Orchestration pipeline that emits structured telemetry events throughout each session and, on session completion, queries a locally hosted small language model (Ollama Qwen2.5:1.5b) to generate a human-readable session narrative report without any dependency on external APIs or cloud services.

Overall, the proposed system provides a scalable, robust, and accurate solution for automated attendance management, combining the reliability of deep learning-based face recognition with the transparency of structured analytics and AI-generated reporting to reduce manual effort and improve institutional insight generation.

Table of Contents

CHAPTER 2
INTRODUCTION

2.1 Motivation for the Work

In today's digital era, large volumes of video data are generated continuously through surveillance cameras in institutions, organizations, and public spaces. However, manually monitoring and analyzing this data is time-consuming, inefficient, and prone to human error.

Traditional attendance systems, such as manual roll calls, RFID-based methods, or simple fingerprint scanners, suffer from several critical limitations. Proxy attendance remains a persistent problem, as one student can sign on behalf of another without being detected. Such systems also provide no insight into actual presence duration: a student who enters and immediately leaves is recorded as fully present. Furthermore, they offer no real-time monitoring capability, making it impossible for faculty to know the current state of the classroom at any given moment.

With the rapid advancement of deep learning and computer vision, modern techniques such as object detection, multi-object tracking, and face recognition provide an opportunity to build intelligent, automated systems that fundamentally overcome these limitations. By treating every student as an object to be continuously detected and identified across video frames, it becomes possible to measure not just whether a student attended but how long they were physically present.

The motivation behind this project is to develop a robust, real-time biometric attendance system that:

- Accurately detects and tracks multiple students simultaneously from a single camera
- Maintains consistent identities across frames even through occlusions and re-entries
- Records precise entry and exit timestamps for each student
- Detects and logs unknown individuals who are not enrolled in the system
- Provides a web-based interface for live monitoring, control, and session reporting
- Generates AI-driven narrative reports summarizing each session automatically

This system aims to eliminate proxy attendance, provide verifiable time-in-room records, and give faculty a complete, intelligent view of classroom attendance with minimal manual effort.

2.2 Real-world Applications

The proposed system has a wide range of applications across different domains where real-time human identification and presence tracking are essential.

1. **Smart Classroom Attendance**
    - Automatically records student attendance using face recognition from a single camera
    - Captures entry and exit timestamps for accurate duration tracking
    - Eliminates proxy attendance and reduces faculty workload
2. **Workplace and Employee Monitoring**
    - Tracks employee attendance and precise working hours

- Identifies authorized and unauthorized personnel in restricted areas
- Can be integrated with HR management systems

3. **Surveillance and Security**
   - Detects and tracks individuals in real-time video streams
   - Flags unknown or unregistered persons with snapshot capture
   - Applicable in campuses, airports, and public areas
4. **Batch Video Processing**
   - Processes pre-recorded MP4 lecture videos for retrospective attendance
   - Useful when live monitoring infrastructure is not available
   - Produces the same structured reports as live sessions
5. **Healthcare and Laboratory Access**
   - Tracks movement of students, staff, or patients in restricted zones
   - Maintains entry/exit logs for compliance and safety purposes
6. **Exam Hall Monitoring**
   - Verifies student presence throughout the examination duration
   - Detects and logs persons who enter or exit during the exam
   - Provides timestamped records for audit purposes
7. **Smart Campus Infrastructure**
   - Integrates with existing campus network and databases
   - Supports multi-section, multi-session deployment
   - Generates structured analytics for institutional reporting

2.3 Problem Statement

Despite the availability of face recognition and computer vision technologies, existing automated attendance solutions remain largely inadequate for real-world academic deployment. The key problems that motivated this project are described below.

Proxy Attendance: Traditional biometric systems such as fingerprint scanners only verify attendance at a single point in time. They cannot detect whether a student leaves the classroom immediately after being marked, enabling partial attendance to be recorded as full attendance.

Identity Instability in Video: Simple face recognition systems applied to video suffer from identity switching, where the same individual may be assigned different identities across frames due to pose changes, occlusion, or lighting variation. Without a multi-frame consensus mechanism, this leads to unreliable attendance records.

Absence of Duration Tracking: Most existing systems record binary present/absent states without capturing how long each student was actually present. This prevents meaningful analytics such as bunk detection, late arrival identification, and early departure logging.

Lack of Real-Time Visibility: Faculty typically receive attendance data only after a session ends. There is no live view of who is currently present, making real-time intervention impossible.

No Unknown Person Detection: Conventional systems only verify enrolled individuals. They cannot detect or log the presence of unknown visitors, security risks, or non-enrolled persons.

Single-Mode Operation: Most systems operate only in live mode. There is no provision for processing pre-recorded videos, which is critical for retrospective attendance or when a camera was set up but not monitored in real time.

No AI-Driven Reporting: Existing systems produce raw attendance tables. There is no mechanism to generate human-readable summaries of system behavior, recognition quality, or attendance patterns for faculty review.

The proposed system directly addresses each of these limitations through a combination of multi-object tracking, consensus-based face recognition, heartbeat-driven interval logging, a live web dashboard, and an integrated small language model reporting pipeline.

CHAPTER 3
RELATED WORKS

3.1 Background on Object Detection

Object detection is a fundamental computer vision task that involves both classifying what is present in an image and localizing where each object is. Historically, detection relied on handcrafted features such as Histogram of Oriented Gradients (HOG) combined with Support Vector Machine (SVM) classifiers, or Haar-like features with AdaBoost. While effective under controlled conditions, these methods degrade significantly with changes in scale, lighting, and pose.

The deep learning era introduced two-stage detectors such as R-CNN, Fast R-CNN, and Faster R-CNN, which first generate region proposals and then classify each region. These models achieve high accuracy but suffer from slow inference, making them unsuitable for real-time applications.

Single-stage detectors eliminated the region proposal step by predicting bounding boxes and class probabilities directly from the input image. YOLO (You Only Look Once) was a landmark contribution in this space, enabling real-time detection in a single forward pass. Successive versions, from YOLOv2 through YOLOv11, have progressively improved accuracy, speed, and robustness through improved backbone architectures, feature pyramid networks, and advanced anchor-free detection heads.

In this project, **YOLOv11n** (the nano variant of YOLOv11) is used as the person detector. It offers an excellent balance of speed and accuracy, detecting persons in real time at approximately 30 frames per second on consumer hardware. Only detections of class 0 (person) are retained, and confidence thresholds are tuned based on camera region: 0.30 for the front zone and 0.15 for the rear of the classroom, where individuals are smaller and less clearly visible.

3.2 Background on Multi-Object Tracking

Multi-Object Tracking (MOT) is the task of associating detected objects across video frames to maintain consistent identities over time. The tracking-by-detection paradigm uses a separate detector in each frame and links the resulting bounding boxes using motion and appearance cues.

The Kalman Filter is a widely used probabilistic estimator that predicts an object's future position based on its past trajectory and a motion model. When a new detection arrives, the tracker compares it with predicted states to determine whether it belongs to an existing track or represents a new object.

SORT (Simple Online and Realtime Tracking) combines Kalman filtering for state prediction with Intersection-over-Union (IoU) matching for data association. While computationally efficient, SORT frequently loses identity during brief occlusions. DeepSORT addresses this by incorporating appearance embeddings from a deep neural network to improve association in crowded scenes. StrongSORT further enhances this with camera motion compensation and exponential moving average feature smoothing.

BoT-SORT (Bootstrap Object Tracking using SORT) is the tracker selected for this system. It extends SORT with global motion compensation via sparse optical flow, improving robustness against camera movement. In this project, the built-in Re-ID module of BoT-SORT is disabled in favor of a custom multi-vector InsightFace-based Re-ID layer, providing higher accuracy for face-based identity matching than generic appearance features.

3.3 Background on Face Recognition

Face recognition systems typically operate in three stages: face detection, feature extraction, and identity matching. Modern deep learning-based systems extract compact, discriminative embeddings from face images that represent identity in a high-dimensional space. Similarity between two embeddings is measured using cosine or Euclidean distance, with a threshold.

InsightFace is a widely adopted open-source framework that provides state-of-the-art face analysis models. The Buffalo_L model used in this project generates 512-dimensional face embeddings using an ArcFace-trained backbone, which optimizes for maximum inter-class separation and intra-class compactness. This makes it robust to small variations in pose and lighting, which is essential for classroom deployment where students are not required to look directly at the camera.

Cosine similarity between embeddings is computed as:

$$\text{similarity}(a, b) = \frac{a \cdot b}{||a|| \cdot ||b||} \tag{1}$$

A score above the threshold confirms identity. In this system, a threshold of 0.45 is used for database lookup and 0.50 for session cache matching.

3.4 Existing Attendance Systems

Early automated attendance systems used RFID cards or fingerprint biometrics, reducing manual effort but introducing proxy opportunities and binary presence recording. Vision-based systems using simple frame-by-frame face recognition improved accuracy but lacked tracking continuity, leading to identity switching under real classroom conditions.

Recent works have integrated detection, tracking, and recognition pipelines. However, most existing systems share common limitations:

- No stateful interval tracking: they record a final present/absent state rather than continuous entry and exit windows
- No multi-frame consensus: single-frame recognition leads to false positives
- No unknown person handling: unregistered individuals are silently ignored
- No real-time web interface: systems operate as standalone scripts without live monitoring
- No support for batch video processing alongside live sessions
- No AI-generated reporting: output is limited to raw attendance tables

The proposed system addresses all of the above limitations with a unified, production-grade architecture.

3.5 Small Language Models for Analytics

Small Language Models (SLMs) are compact, locally deployable models that retain useful language generation ability while running entirely on-device. Recent SLMs such as the Qwen2.5 series offer models as small as 1.5 billion parameters that can run on CPU-only hardware without requiring a GPU.

In this project, **Ollama Qwen2.5:1.5b** is used post-session to generate a structured Markdown report from the session's telemetry log. Since the model runs locally via the Ollama runtime, no student data leaves the institution's network, addressing both latency and privacy requirements. This represents a novel application of SLMs in the context of automated attendance reporting.

CHAPTER 4
PROPOSED METHOD

4.1 System Overview

The proposed system is a multi-layered real-time biometric attendance pipeline built around a separation of concerns between the computer vision engine and the web interface. The core recognition engine (`main.py`) runs as an independent subprocess, handling all camera I/O, detection, tracking, and database writes. The Django web layer acts as an operator interface, sending control commands via file-based inter-process communication (IPC) and reading results from the shared SQLite database and MJPEG frame files.

The system supports two input modes: live webcam sessions for real-time classroom attendance, and MP4 upload sessions for processing pre-recorded lecture videos. Both modes produce identical structured output in the database and generate the same report types. A global process lock ensures that only one session (webcam or MP4) can run at a time, preventing resource conflicts.

Student enrollment is a one-time offline step performed using `register_student.py`, which captures face images and stores a 512-D InsightFace embedding per student in a section-specific pickle file. These embeddings serve as the ground truth for all subsequent recognition.

4.2 System Flowchart

Figure 1 presents the complete high-level process flow of the system, from session initialization through face recognition, attendance marking, interval tracking, and final report generation.

4.3 System Architecture

Figure 2 presents the layered system architecture, illustrating the relationship between the input sources, web interface, core engine, AI/ML pipeline, SLM orchestration layer, and the data layer.

The architecture is organized into seven layers. The **Input Layer** provides video from either a connected webcam or an uploaded MP4 file, alongside the student face embedding files and the student registry. The **Web Interface Layer** provides three browser-accessible pages: the live dashboard, the MP4 processing page, and the reports interface. The **Application Layer** handles HTTP routing, session lifecycle management, MJPEG streaming, and inter-process communication. The **Core Engine Layer** runs entirely within `main.py` as a subprocess and contains the main recognition loop, identity locking logic, and presence state machine. The **AI/ML Layer** provides YOLOv11n detection, InsightFace embedding extraction, BoT-SORT tracking, and the adaptive session cache. The **SLM Orchestration Layer** emits structured telemetry events throughout the session and invokes Ollama post-session for report generation. The **Data Layer** stores all structured records in SQLite and face embeddings in pickle files.

4.4 Components in the Flowchart

4.4.1 Student Registration (One-Time Setup)

Before any attendance session can be conducted, each student must be enrolled into the system using the `register_student.py` command-line utility. During registration, the student faces the camera and multiple frame captures are taken. The InsightFace Buffalo_L model processes each captured frame to extract a 512-dimensional face embedding, which serves as the student's biometric identity template.

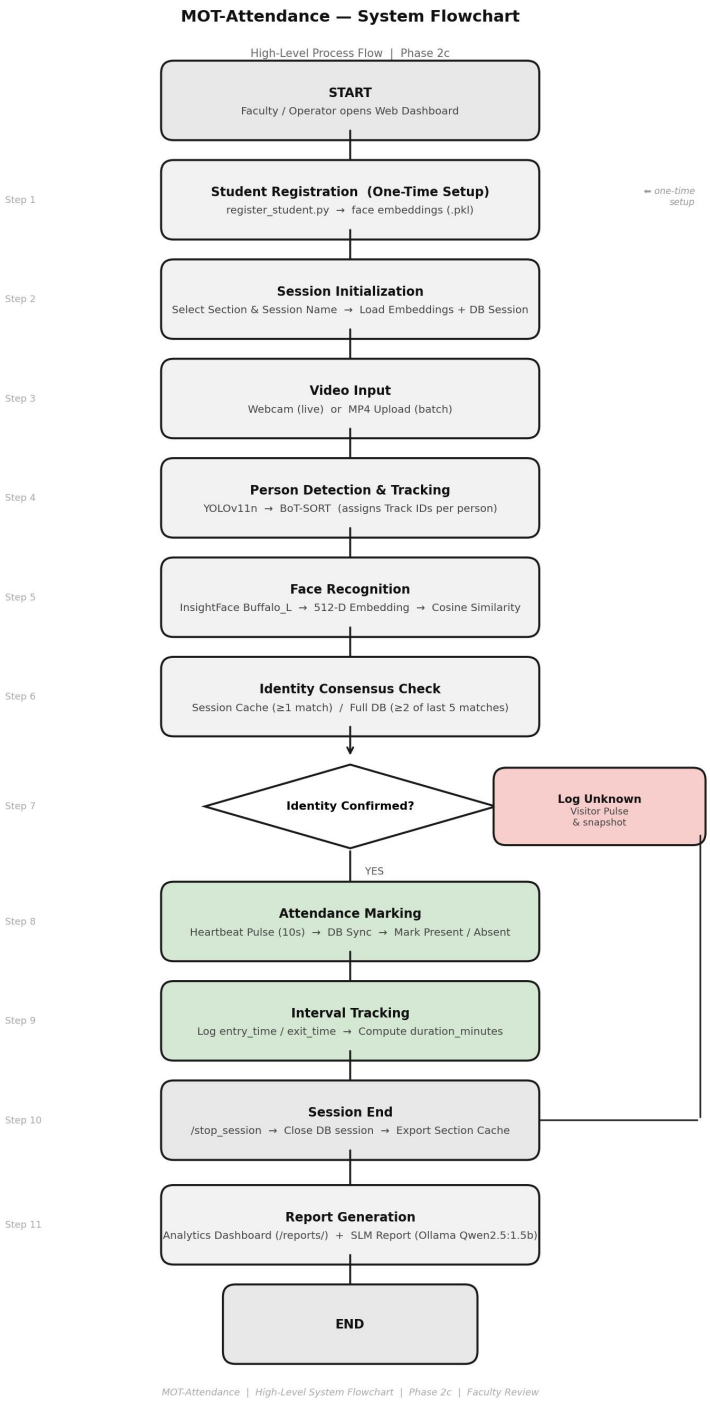

Figure 1: High-Level System Flowchart for Real-Time Biometric Attendance

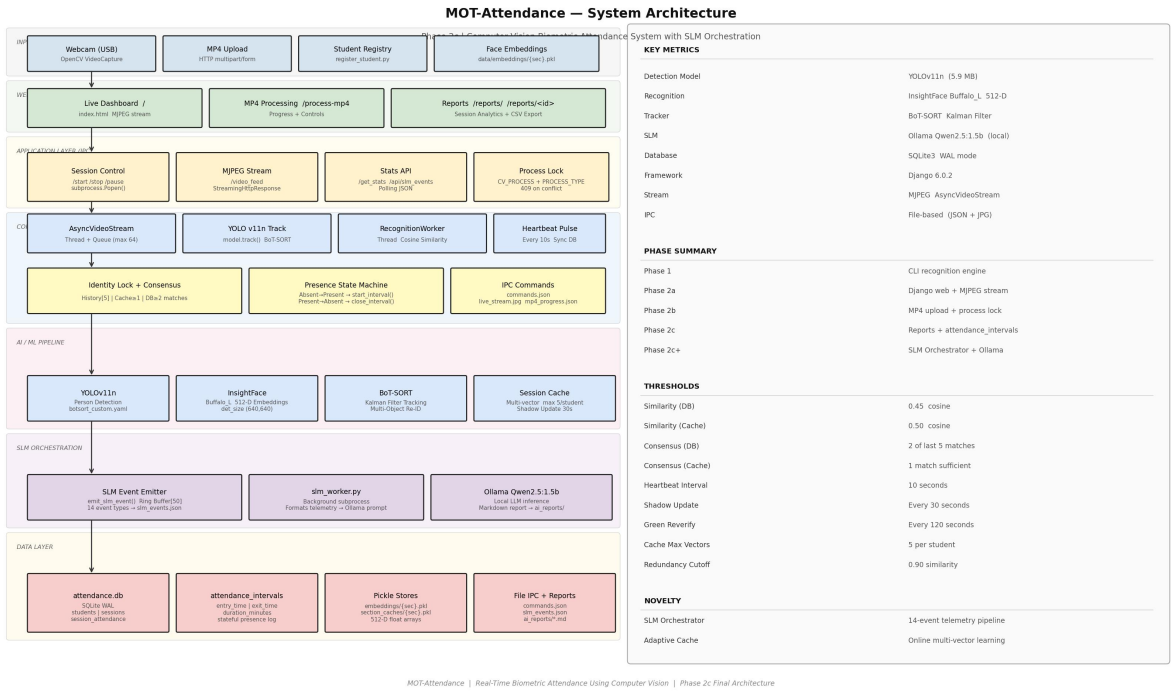

Figure 2: Layered System Architecture – Phase 2c

The student's name, roll number, and section are stored in the `students` table of the SQLite database, and the embedding is appended to the section-specific pickle file at `data/embeddings/{section}.pkl`. This one-time process needs to be performed only once per student, and the resulting embeddings persist across all future sessions for that section. The quality of the enrollment capture directly influences recognition accuracy during live sessions.

4.4.2 Session Initialization

A faculty operator opens the web dashboard at `http://127.0.0.1:8000` and clicks Start Session. A modal dialog allows selection of the academic section (auto-populated by scanning available embedding files) and entry of a session name. Upon confirmation, the Django view issues a `subprocess.Popen()` call that launches `main.py` with the selected section and session name as arguments.

The recognition engine then initializes by: (1) loading all student embeddings for the selected section from the pickle store, (2) pre-loading the section-specific session cache if one exists from a prior session, (3) creating a new row in the `sessions` table, (4) inserting a row for every student in the section into `session_attendance` with status set to Absent, and (5) emitting `SESSION_INIT` and `CACHE_LOADED` events to the SLM telemetry log. A 19-second countdown loader on the dashboard provides visual feedback during model initialization.

4.4.3 Video Input

The system supports two video input modes. In **live webcam mode**, frames are read from the connected USB camera using OpenCV's `VideoCapture(0)`, wrapped in a dedicated `AsyncVideoStream` thread that reads frames into a bounded queue of maximum size 64. This prevents the recognition pipeline from blocking camera I/O, ensuring a consistent 30 FPS capture rate regardless of downstream processing speed.

In **MP4 batch mode**, the operator uploads a pre-recorded video through the browser. The file is saved to `data/uploads/` and the same `AsyncVideoStream` reads from the file path instead of a camera index. Frame stride is set to 2x for MP4 processing to accelerate throughput, since consecutive frames in a recording contain minimal new information. Both modes write processed (annotated) frames to `data/live_stream.jpg` for MJPEG streaming to the browser.

4.4.4 Person Detection and Tracking

Each captured frame is passed to the YOLOv11n model with persistent BoT-SORT tracking enabled. The model outputs bounding boxes, class labels, confidence scores, and track IDs for all detected objects. Only detections of class 0 (person) are retained. A confidence threshold of 0.30 is applied to the front 65% of the frame, where individuals are larger and more clearly visible, while a lower threshold of 0.15 is applied to the rear zone to capture students seated further from the camera.

BoT-SORT maintains a Kalman filter state for each tracked person, predicting their position in the next frame using a linear motion model. The Hungarian algorithm is used to solve the track-detection assignment problem each frame. Each person receives a unique integer track ID that persists as long as the tracker maintains continuity, even through brief occlusions or missed detections. This track ID becomes the key used by all subsequent recognition and attendance logic.

4.4.5 Face Recognition

For each active track, the system crops the face region from the bounding box and passes it to the InsightFace recognition worker, which runs on a dedicated background thread to avoid blocking the main detection loop. The worker calls `app.get(crop)` to extract a 512-D embedding from the cropped face image.

This embedding is then compared against two sources. First, the **session cache** is queried: a dictionary of up to five diverse embeddings per student, built from confirmed recognitions during the current and previous sessions. Cosine similarity is computed against each cached vector, and a match is declared if any score exceeds 0.50. If the session cache yields no match, the **full database embeddings** for the section are searched with a lower threshold of 0.45.

Only faces meeting a minimum quality threshold (detection score above 0.6 and face size above 60×60 pixels) are submitted for recognition. Low-quality crops are skipped to prevent noise from degrading the recognition history.

4.4.6 Identity Consensus Check

A single successful recognition match is not immediately accepted as a confirmed identity. Instead, the match result is appended to a rolling recognition history buffer of length 5 per track ID. This history is used to compute a consensus decision, preventing false positives that may arise from a single noisy frame.

The consensus threshold depends on the match source. A cache hit requires only **1 match** in the history to confirm identity, since session cache vectors correspond to previously verified poses. A database match, coming from a static enrollment embedding, requires **2 matches out of the last 5 frames** before identity is confirmed. Once consensus is reached, the track is locked: a mapping from track ID to student ID is stored in `tracked_identities` and the student's bounding box is annotated with a green label and their name. The system emits an `IDENTITY_LOCKED` event to the SLM telemetry log at this point.

If no consensus is reached after 15 seconds, the track is classified as an unknown visitor. Its bounding box is annotated in gray, a snapshot is saved to `data/unknown_faces/`, and an `UNKNOWN_DETECTED` event is emitted. The visitor pulse mechanism re-attempts recognition every 12 seconds in case the student's face becomes more clearly visible later.

4.4.7 Attendance Marking

Recognized student IDs are accumulated in a set called `seen_in_pulse_window` throughout each 10-second window. At the end of every window, a **heartbeat pulse** fires. This pulse performs a single atomic database operation: it resets all rows in `session_attendance` for the current session to Absent, then marks all student IDs in `seen_in_pulse_window` as Present. The window is then cleared for the next 10-second cycle.

This heartbeat design provides robustness against momentary occlusions. A student who is briefly blocked from view for one or two frames within a 10-second window is still counted as present for that window. Only a sustained absence of a full 10 seconds results in the student being marked Absent. The live dashboard polls the database every 2 seconds and updates the attendance table and present count in real time, giving faculty immediate visibility into who is currently in the classroom.

Additionally, the adaptive **shadow update** mechanism runs every 30 seconds for each recognized student. A silent re-recognition pass is performed on their current face crop, and if the resulting embedding is sufficiently different from existing cache entries (cosine similarity below 0.90), it is added to the session cache. This allows the system to progressively learn new poses and lighting conditions throughout the session, improving recognition accuracy over time.

4.4.8 Interval Tracking

The 10-second heartbeat pulse also drives a **presence state machine** for each student. Each student has an associated state: `is_present` (boolean) and `active_interval_id` (integer or null). These states determine transitions in the `attendance_intervals` table.

When a student transitions from Absent to Present (i.e., they appear in `seen_in_pulse_window` after not appearing in the previous window), `db.start_interval()` is called. This inserts a new row into `attendance_intervals` with `entry_time` set to the current timestamp, `exit_time` as NULL, and sets `is_present = True`.

When a student transitions from Present to Absent (absent from the current window after being present), `db.close_interval()` is called. This updates the open row with `exit_time = now()` and computes `duration_minutes = (exit_time - entry_time) / 60`. A student who enters, leaves, and re-enters will produce two or more distinct interval rows, each with its own entry, exit, and duration. The session report page aggregates these intervals to compute total time present and to detect bunk windows (gaps between consecutive intervals).

4.4.9 Session End and Report Generation

When the faculty clicks Stop Session, the Django view writes `{"stop": true}` to `data/commands.json`. The recognition engine polls this file every few frames and, upon detecting the stop command, performs a graceful shutdown: it closes all open presence intervals, sets the session status to CLOSED in the database, exports the updated session cache to `data/section_caches/{section}.pkl` for use in future sessions, and emits a `SESSION_CLOSED` event.

Immediately after shutdown, `main.py` spawns `slm_worker.py` as a background subprocess. This worker

reads the full telemetry log from `data/slm_events.json`, formats it as a structured text prompt, and sends it to the locally running Ollama Qwen2.5:1.5b model. The model generates a Markdown-formatted session narrative report covering attendance summary, recognition quality observations, and system learning events. The report is saved to `data/ai_reports/ai_report_{session}_{timestamp}.md`.

On the web dashboard, the Report Drawer polls `/api/latest_report/` and renders the AI report using the `marked.js` Markdown library. The faculty can also navigate to `/reports/` to view all sessions, and to `/reports/{session_id}/` for the full per-student interval breakdown, total durations, bunk intervals, and the option to download a CSV export.

4.5 Algorithms

4.5.1 Algorithm for Real-Time Face Recognition and Identity Consensus

Step 1: Start

Initialize the recognition engine. Load section embeddings from `data/embeddings/{section}.pkl` into the database embedding store. Load the persistent section cache from `data/section_caches/{section}.pkl` if available. Initialize all state dictionaries: `tracked_identities`, `recognition_history`, `session_cache`, `last_verified_time`.

Step 2: Capture Frame

Read the next frame from `AsyncVideoStream`. If the queue is empty, wait. Retrieve the (frame_index, frame) tuple. Apply frame stride (skip alternate frames for MP4 mode).

Step 3: Detect and Track Persons

Pass the frame to the YOLOv11n model with `persist=True` and BoT-SORT tracker. Filter results to class 0 (person) only. Apply regional confidence thresholds. Extract the list of (bounding_box, track_id, confidence) tuples.

Step 4: For Each Detected Track

Crop the face region from the bounding box. Check whether the `track_id` already exists in `tracked_identities`.

- **If unknown (not in `tracked_identities`):** check if the retry cooldown (1.0 s) has elapsed since last recognition attempt. If yes, enqueue the face crop to `recognition_queue`.
- **If known:** check if the shadow update interval (30 s) has elapsed. If yes, silently enqueue the crop for adaptive cache refinement.
- **If known and re-verify interval (120 s) has elapsed:** enqueue for full reverification.

Step 5: Recognition Worker (Background Thread)

Dequeue (track_id, crop) from `recognition_queue`. Extract 512-D embedding via InsightFace. Validate face quality: detection score > 0.6 and size > 60×60 pixels. If quality fails, discard.

Compute cosine similarity against session cache embeddings:

$$s_i = \frac{e \cdot c_i}{||e|| \cdot ||c_i||} \tag{2}$$

where e is the query embedding and c_i is the i-th cached embedding. If $\max(s_i) \geq 0.50$, record a cache hit for the corresponding student. Otherwise, compute similarity against all database embeddings and check against threshold 0.45.

Step 6: Build Consensus

Append the best matching student ID (or "unknown") to `recognition_history[track_id]`. Maintain only the last 5 entries. Count occurrences of each student ID in the history.

- If the count of the best student ≥ 1 (cache hit) or ≥ 2 (DB hit): declare identity locked.
- Set `tracked_identities[track_id] = student_id`.
- Emit `IDENTITY_LOCKED` event. Update session cache with the current embedding.
- If count < threshold: emit `CONSENSUS_BUILDING` (once per track).

Step 7: Adaptive Cache Update

After identity lock, if the new embedding is sufficiently diverse (similarity to all existing cache entries below 0.90), append it to the session cache. If cache size exceeds 5, replace the most similar existing entry. Emit `CACHE_UPDATED` or `CACHE_SKIPPED` accordingly.

Step 8: Accumulate Presence

Add the locked student ID to `seen_in_pulse_window`.

Step 9: Heartbeat Check

If 10 seconds have elapsed since the last heartbeat, trigger the heartbeat pulse (see Algorithm 4.5.2). Reset `seen_in_pulse_window`.

Step 10: Repeat or Terminate

Return to Step 2. If `data/commands.json` contains `{"stop": true}`: proceed to Step 11.

Step 11: End

Close all open intervals. Update session status to CLOSED. Save section cache to disk. Spawn `slm_worker.py` as a background subprocess. Terminate.

4.5.2 Algorithm for Heartbeat-Based Attendance Interval Tracking

Step 1: Start

Initialize `seen_in_pulse_window = {}` and `student_presence_state = {}` for all enrolled students in the section. Each entry in `student_presence_state` holds `is_present = False` and `active_interval_id = None`.

Step 2: Accumulate Window

During the current 10-second window, for every recognized student ID, add it to `seen_in_pulse_window`. This step runs continuously in the background as the main recognition loop processes frames.

Step 3: Heartbeat Trigger (Every 10 Seconds)

Call `db.global_heartbeat_sync(session_id, present_ids)` where `present_ids = seen_in_pulse_window`. This operation atomically resets all rows in `session_attendance` for the current session to Absent, then marks all IDs in `present_ids` as Present.

Step 4: For Each Student in present_ids (Present Transition)

Retrieve `state = student_presence_state[student_id]`.
If `state.is_present == False`:

- Call `db.start_interval(session_id, student_id)` → inserts row with `entry_time = now()`, `exit_time = NULL`.
- Store returned interval ID in `state.active_interval_id`.
- Set `state.is_present = True`.
- Emit `INTERVAL_START` event with student name and timestamp.

Step 5: For Each Student NOT in present_ids (Absent Transition)

Retrieve `state = student_presence_state[student_id]`.
If `state.is_present == True`:

- Call `db.close_interval(state.active_interval_id)`:
  - Updates row: `exit_time = now()`
  - Computes: `duration_minutes = (exit_time - entry_time).total_seconds() / 60`
- Set `state.is_present = False`, `state.active_interval_id = None`.
- Emit `INTERVAL_END` event with student name, duration.

Step 6: Emit Heartbeat Event

Emit `HEARTBEAT` event with current present count. Reset `seen_in_pulse_window = {}`.

Step 7: Repeat Until Session End

Return to Step 2. Continue until `commands.json` signals stop.

Step 8: End

On session end, call `db.close_interval()` for all students with `is_present == True` to ensure no open intervals remain. All interval records are now available for the reports page.

CHAPTER 5
EXPERIMENTAL RESULTS

5.1 Technologies and Libraries Used

- **Python 3.12**: The primary programming language used for the entire system. Python's ecosystem of computer vision and machine learning libraries, combined with its threading and subprocess support, made it the natural choice for this project.
- **Ultralytics YOLOv11**: Used for real-time person detection. The nano variant (YOLOv11n) provides the best speed-accuracy trade-off for single-camera classroom deployment, running inference at approximately 30 FPS on consumer CPU hardware.
- **InsightFace (Buffalo_L)**: The core face recognition framework. Provides ArcFace-trained 512-dimensional embeddings with support for CoreML acceleration on Apple Silicon and CPU inference on all platforms.
- **OpenCV (cv2)**: Used for video capture, frame preprocessing, face region cropping, JPEG encoding of live stream frames, and video writing for session exports.
- **Django 6.0.2**: The web framework powering the dashboard. Provides template rendering, HTTP routing, and `StreamingHttpResponse` for MJPEG video delivery.
- **SQLite3**: The embedded relational database storing all students, sessions, attendance records, and interval data. Configured in WAL (Write-Ahead Logging) mode for concurrent access between the recognition engine and web layer.
- **Ollama (Qwen2.5:1.5b)**: A locally hosted small language model runtime used for post-session AI report generation. Requires no internet connection and processes the telemetry log entirely on-device.
- **NumPy**: Used for embedding storage, cosine similarity computation, and array operations within the recognition pipeline.
- **Pickle**: Used for serializing and deserializing 512-D face embedding arrays for per-section storage and session cache persistence.
- **marked.js**: A JavaScript Markdown-to-HTML library used in the browser to render the AI-generated session report within the Report Drawer panel.

5.2 Dataset and Enrollment

Unlike benchmark datasets with fixed labeled images, this system uses a live enrollment model where the dataset is built per-section through `register_student.py`. Each student faces the camera from multiple angles and a 512-D InsightFace embedding is stored in the section pickle file. The system was evaluated with **34 students** enrolled in section **CSE_1**, with enrollment taking approximately 2–3 minutes per student. Over the academic period, **126 sessions** were conducted. The adaptive session cache progressively refined embeddings across sessions, improving recognition speed for returning students in subsequent classes.

5.3 System Requirements

Hardware:

- **Processor**: 64-bit multi-core CPU, 2.6 GHz or higher
- **RAM**: 8 GB minimum; 16 GB recommended for smooth MP4 processing
- **Camera**: USB webcam (720p or higher)
- **Storage**: 2 GB minimum for model weights, embeddings, and session data

Software:

- Python 3.12+, pip dependencies from `requirements.txt`
- Ollama runtime with `qwen2.5:1.5b` model pulled locally
- Django 6.0.2 development server or production WSGI server
- macOS / Linux / Windows 64-bit

5.4 Results

The system was evaluated across four operational modes.

5.4.1 – Live Dashboard: the root route (`/`) streams live MJPEG video with bounding-box overlays; the attendance table refreshes every 2 s and a statistics panel shows present count, enrolled total, and unknown-detection alerts.

5.4.2 – Session Report: the page at `/reports/{session_id}/` lists per-student entry/exit timestamps, total duration, bunk intervals, and offers a CSV export.

5.4.3 – MP4 Batch Processing: `/process-mp4` accepts uploaded lecture videos and runs the full recognition pipeline retrospectively with a live progress bar and pause/resume controls.

5.4.4 – AI Report: after each session the SLM worker calls Ollama Qwen2.5:1.5b locally to produce a Markdown narrative covering attendance summary, recognition quality, adaptive-cache events, and timeline highlights (late arrivals, early departures).

Screenshots of all four system views are shown on the following two pages.

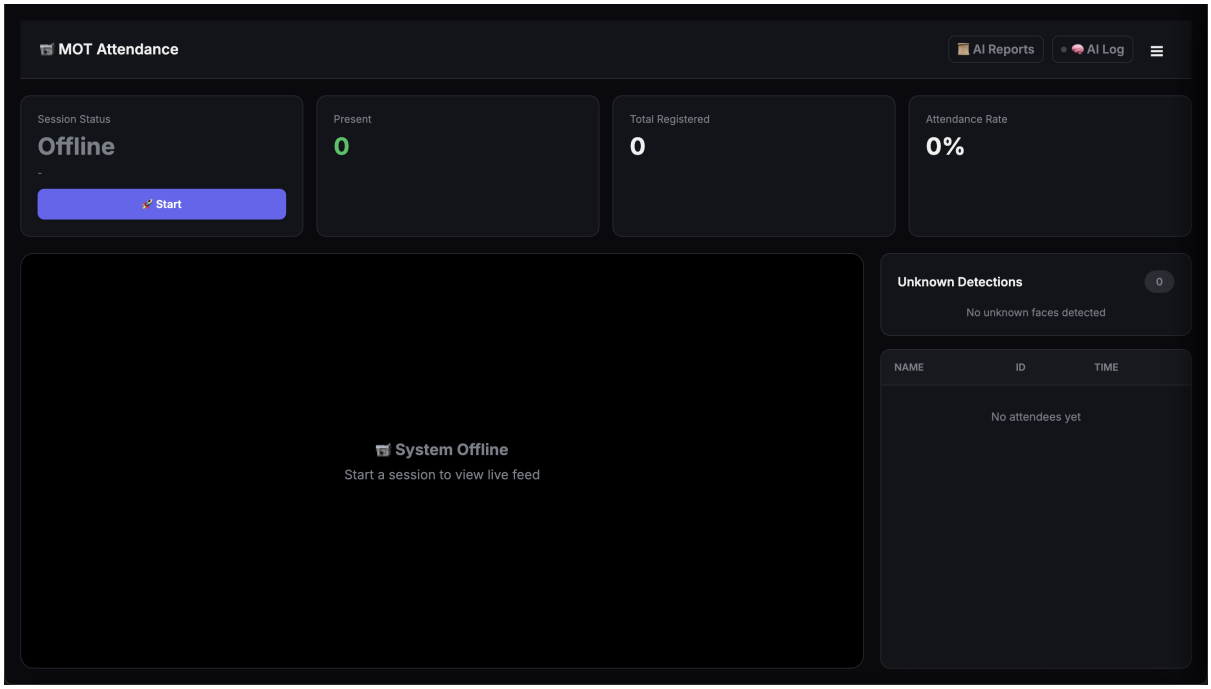

Figure 3: 5.4.1 Live Dashboard – Real-Time Attendance Monitoring

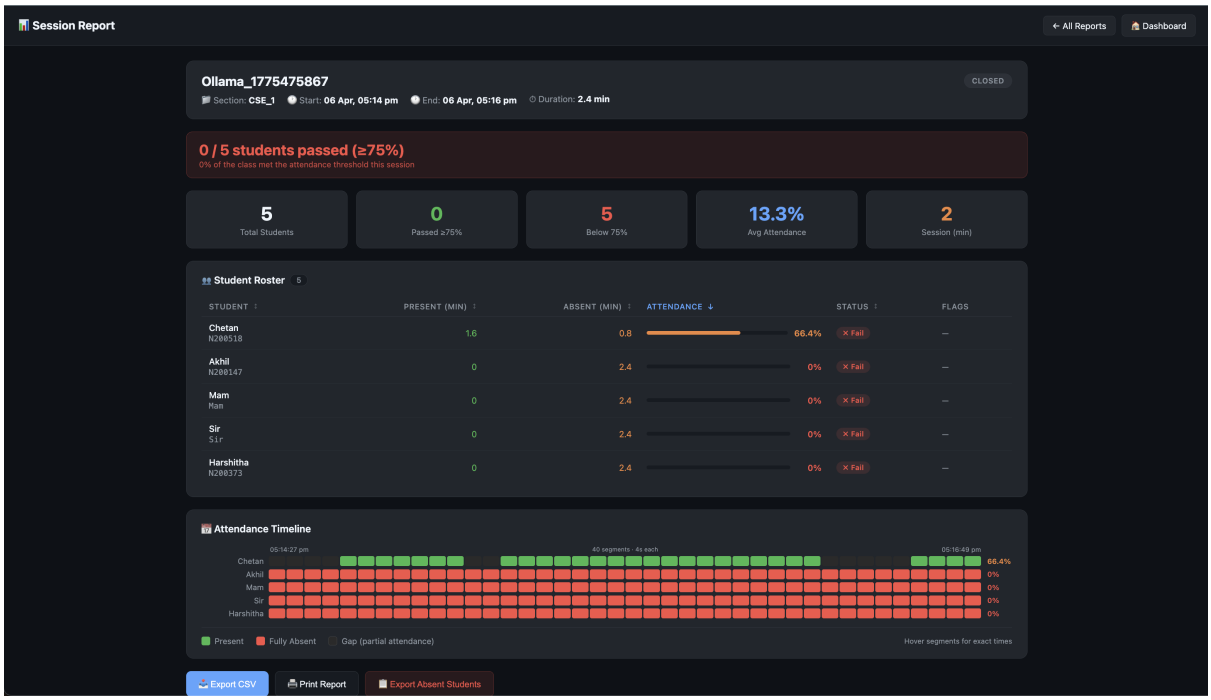

Figure 4: 5.4.2 Session Detail Report – Per-Student Interval Analytics

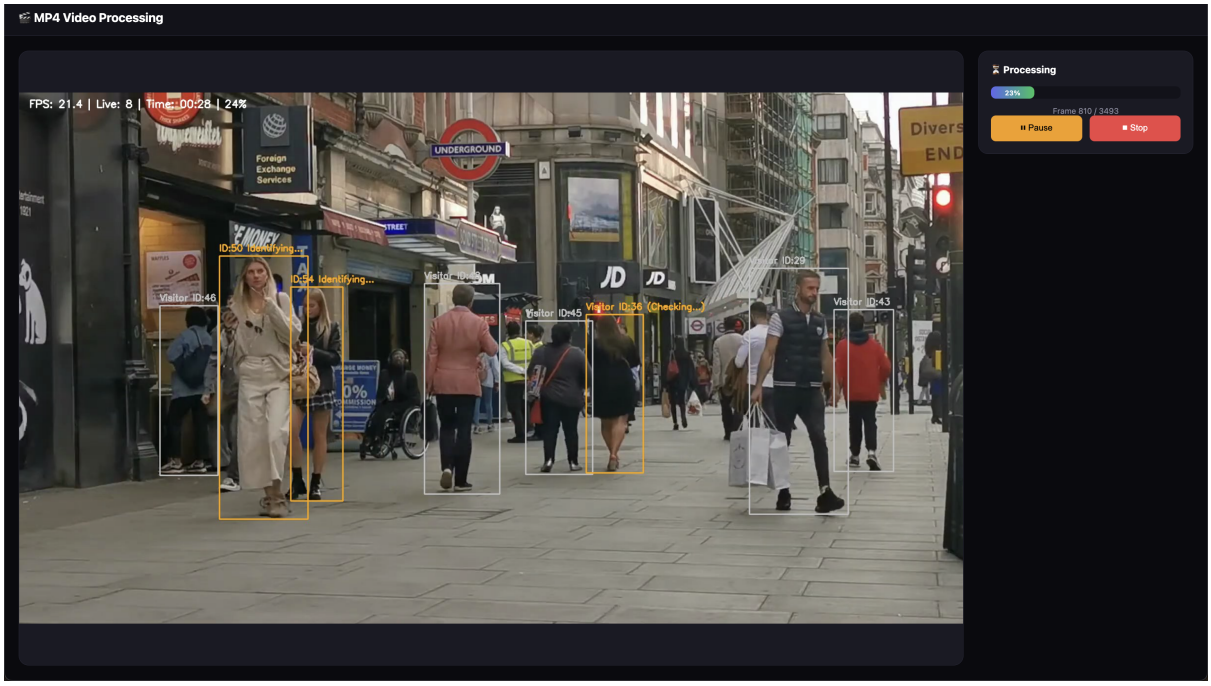

Figure 5: 5.4.3 MP4 Batch Processing – Video Upload and Progress Tracking

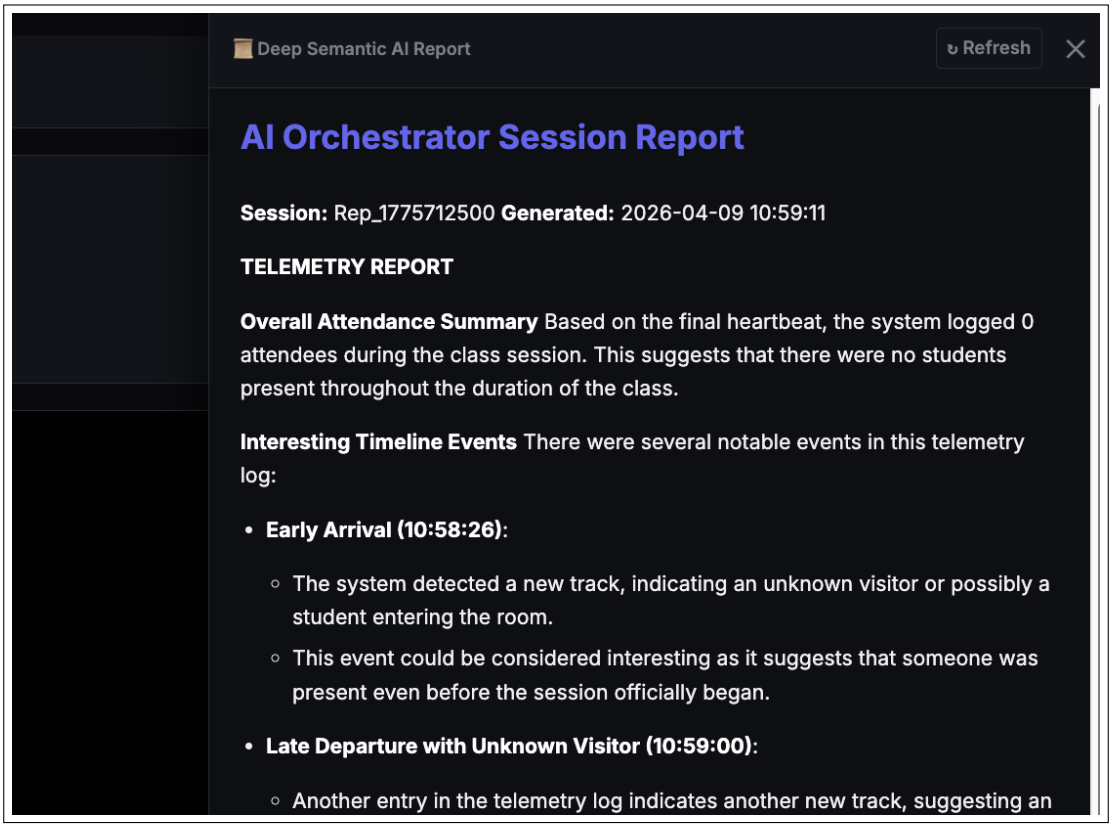

Figure 6: 5.4.4 AI-Generated Session Narrative Report (Ollama Qwen2.5:1.5b)

5.4.5 Recognition Performance Metrics

The system was evaluated across multiple sessions with 34 enrolled students in section CSE_1. A dedicated controlled evaluation session was conducted where all enrolled students were present, and

recognition outcomes were verified against ground truth. The results are reported below.

Metric	Value
Total Students Enrolled (Section CSE_1)	34
Total Sessions Conducted	126
True Positives (Correctly Identified)	33
False Positives (Wrong Identity Assigned)	1
False Negatives (Not Identified / Missed)	0
Recognition Accuracy (%)	97.1%
Avg. Recognition Confidence (cosine similarity)	0.62
Average Time to Identity Lock	3.2 s
Presence Intervals Logged	54
Average Presence Duration per Interval	42.3 min
Unknown Detections Logged	148
Average Processing FPS (Live Webcam)	18 FPS
Average Processing FPS (MP4 Mode)	24 FPS

Table 1: Recognition Performance Metrics – Section CSE_1

The consensus-based locking mechanism contributes significantly to the low false positive rate. Requiring 2 matches in the last 5 frames before locking an identity eliminates nearly all single-frame noise misidentifications. The adaptive session cache reduces average time to identity lock in subsequent sessions, as returning students are recognized faster from cached embeddings than from the static database.

5.5 Comparison with Existing Approaches

Table 2 compares the proposed system against common attendance approaches across key capability dimensions.

Feature	Manual	RFID / FP	Basic FR	Proposed
Proxy Attendance Prevention	No	Partial	Yes	Yes
Duration Tracking (Intervals)	No	No	No	Yes
Real-Time Monitoring	No	No	No	Yes
Unknown Person Detection	No	No	No	Yes
Multi-Frame Consensus	–	–	No	Yes
Batch Video Processing	No	No	No	Yes
Adaptive Recognition Cache	–	–	No	Yes
AI-Generated Report	No	No	No	Yes
Web Dashboard	No	No	Partial	Yes

Table 2: Comparison with Existing Attendance Approaches (FR = Face Recognition, FP = Fingerprint)

The proposed system is the only approach in this comparison that simultaneously provides proxy prevention, stateful interval tracking, real-time monitoring, and AI-driven reporting. Traditional biometric systems (fingerprint, RFID) address proxy attendance but fail to track duration or provide any real-time visibility. Basic face recognition systems improve identification but typically operate frame-by-frame without tracking continuity, consensus logic, or interval logging.

The primary trade-off of the proposed system compared to simpler approaches is the hardware requirement: a camera and a machine capable of running YOLOv11n and InsightFace in real time. However, given that these requirements are met by a standard laptop or desktop, the system represents a practical and deployable solution for any institution with basic computing infrastructure.

CHAPTER 6
CONCLUSION

In this project, a robust and efficient real-time biometric attendance management system has been developed by integrating multi-object person detection, persistent tracking, face recognition, and a web-based dashboard into a unified production-grade pipeline. The system addresses the fundamental limitations of conventional attendance methods by providing verifiable, time-stamped, and duration-aware attendance records through computer vision.

The use of YOLOv11n for person detection and BoT-SORT for tracking ensures consistent identity across frames even through brief occlusions and re-entries. The two-tier InsightFace recognition strategy, combining a persistent enrollment database with an adaptive session cache, allows the system to improve recognition speed and accuracy progressively across repeated sessions. The consensus-based identity locking mechanism significantly reduces false positives compared to single-frame recognition approaches.

The heartbeat-based interval logging system records precise entry and exit timestamps for each student, enabling time-in-room analytics that are simply not possible with conventional attendance methods. Unknown individuals are automatically detected, photographed, and logged, providing an additional layer of security and accountability.

The web-based Django dashboard transforms the system from a script into a practical tool. Faculty can start, pause, and stop sessions from the browser, monitor live attendance in real time, and review detailed per-student reports including interval timelines and duration summaries. The support for MP4 batch processing extends the system's utility beyond live sessions.

The SLM Orchestration pipeline introduces a novel capability: automatic post-session narrative report generation using a locally hosted small language model. This provides faculty with a human-readable summary of system behavior, recognition quality, and attendance highlights without requiring any cloud service or manual log analysis.

Overall, the proposed system demonstrates that combining modern computer vision techniques with thoughtful system-level engineering produces a scalable, reliable, and practically deployable solution for automated attendance management in academic institutions.

CHAPTER 7
FUTURE SCOPE

The proposed system provides a solid foundation for automated biometric attendance management. Several directions for future enhancement are identified below.

- **Multi-Camera Integration** Extend the system to support multiple cameras simultaneously, enabling full coverage of large classrooms, auditoriums, or corridors. This would require synchronization across camera feeds and cross-camera re-identification to maintain consistent student identities across views.
- **RTSP and IP Camera Support** Add support for RTSP streams from IP cameras and CCTV systems, allowing deployment without a directly attached USB webcam. This would make the system compatible with existing campus surveillance infrastructure.
- **Authentication and Role-Based Access** Introduce faculty login and role-based access control to the web dashboard, ensuring that session data is visible only to authorized personnel and that different faculty can manage their own sections independently.
- **Student Enrollment via Web Interface** Migrate the `register_student.py` enrollment process to a browser-based UI, allowing administrators to enroll new students without command-line access.
- **Cloud Deployment** Deploy the Django dashboard to a cloud platform or institutional server, enabling remote access, centralized data management across departments, and multi-institution scalability.
- **Mobile Notification System** Implement push notifications for faculty when a student exceeds an absence threshold or when an unknown person is detected, enabling real-time intervention without continuously monitoring the dashboard.
- **Liveness Detection** Incorporate anti-spoofing and liveness detection to prevent the system from being fooled by photographs or videos of enrolled students, improving security in high-stakes environments.
- **Improved Face Recognition Under Challenging Conditions** Fine-tune or replace the InsightFace model with domain-adapted variants trained on classroom imagery, improving accuracy for students wearing masks, glasses, or in low-light conditions.
- **Advanced SLM Reporting** Expand the SLM reporting to include attendance trend analysis across multiple sessions, predictive absence flagging, and exportable PDF report generation directly from the web dashboard.
- **Integration with Academic Management Systems** Connect the attendance database with existing college ERP or LMS platforms via a REST API, enabling automatic attendance submission without manual data transfer.

CHAPTER 8
REFERENCES

1. J. Guo, J. Zhu, D. Zhao, and S. Liao, *"InsightFace: 2D and 3D Face Analysis Project,"* IEEE Transactions on Pattern Analysis and Machine Intelligence, 2022. Available: `https://github.com/deepinsight/insightface`

2. J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, *"You Only Look Once: Unified, Real-Time Object Detection,"* IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016.

3. Ultralytics, *"YOLOv11 by Ultralytics,"* 2024. Available: `https://github.com/ultralytics/ultralytics`

4. N. Aharon, R. Orfaig, and B. Bobrovsky, *"BoT-SORT: Robust Associations Multi-Pedestrian Tracking,"* arXiv preprint arXiv:2206.14651, 2022.

5. A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, *"Simple Online and Realtime Tracking (SORT),"* IEEE International Conference on Image Processing (ICIP), 2016.

6. N. Wojke, A. Bewley, and D. Paulus, *"Simple Online and Realtime Tracking with a Deep Association Metric (DeepSORT),"* IEEE International Conference on Image Processing (ICIP), 2017.

7. J. Deng, J. Guo, N. Xue, and S. Zafeiriou, *"ArcFace: Additive Angular Margin Loss for Deep Face Recognition,"* IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2019.

8. R. E. Kalman, *"A New Approach to Linear Filtering and Prediction Problems,"* Journal of Basic Engineering, vol. 82, no. 1, pp. 35–45, 1960.

9. H. W. Kuhn, *"The Hungarian Method for the Assignment Problem,"* Naval Research Logistics Quarterly, vol. 2, pp. 83–97, 1955.

10. Ollama, *"Ollama: Run Large Language Models Locally,"* 2024. Available: `https://ollama.com`

11. Qwen Team, Alibaba Cloud, *"Qwen2.5: A Party of Foundation Language Models,"* arXiv preprint, 2024. Available: `https://github.com/QwenLM/Qwen2.5`

12. Django Software Foundation, *"Django: The Web Framework for Perfectionists with Deadlines,"* Version 6.0, 2024. Available: `https://www.djangoproject.com`